LifeDrop

Audit Prompts

BLOOD DONATION NETWORK

Build prompts 0 se 17 (LifeDrop_Prompts.md) ke hisaab se bane audit prompts.
Ek-ek prompt copy karke Antigravity me bhejo.

Dark edition | 21 Oct 2026

Kaise use karna hai

Ye audit prompts sirf check karte hain, kuch badalte nahi. Fix tab hota hai jab tum report approve karo (Prompt F1).

1. **Pehle git branch banao:** git checkout -b audit/2026-10-21 (apni date). Audit kabhi main par nahi.
2. **Alag database banao:** PostgreSQL me CREATE DATABASE lifedrop_audit; Tests aur e2e script isi par chalenge, tumhare asli lifedrop database par nahi.
3. **A0 sabse pehle bhejo**, usme <N> ki jagah apna last finished build prompt number likho. Phir neeche wale table ke hisaab se agle prompts ek-ek karke.
4. **Har audit ke baad** AUDIT_REPORT.md khud padho. NOT-BUILT-YET wali cheezein bug nahi hain, wo tumhare agle build prompts hain.
5. **Sabse aakhir me A10** chalao aur report approve karne ke baad hi F1 (fix) use karo.

Kab kaun sa prompt chalana hai

Kab	Prompts	Area
Abhi (backend 1-8 ban chuka)	**A0, A1, A2, A3, A4, A5**	Backend
Build prompt 10 ke baad	**A7 (web + UI)**	Web
Build prompt 14 ke baad	**A6 (mobile)**	Mobile
Build prompt 15 ke baad	**A8 (end-to-end, SOS ke saath)**	Full-stack
Build prompt 17 ke baad	**A9, phir A10**	Release
Report approve karke	**F1, phir R1**	Fix

Time kam ho to sirf QUICK AUDIT (ek prompt) use kar sakte ho, lekin wo sirf jaldi wali screening hai.

Status aur severity

Status	Matlab
VERIFIED	Chala ke ya test se pass hua
FAILED	Chalaya aur fail hua
CODE-REVIEW-ONLY	Code padha par chala nahi paya
NOT-VERIFIED	Check nahi ho paya (reason likha hoga)
MISSING	Tumhare build stage par hona chahiye tha, par nahi hai
NOT-BUILT-YET	Aage ke build prompt ka kaam hai, bug nahi

Severity	Matlab
P0	Security hole, data loss, ya core flow toota hua
P1	Main flow me important bug ya missing feature
P2	Chhota bug, quality, polish
P3	Suggestion

Project me jo cheezein hain hi nahi (report me missing nahi aani chahiye)

OTP / email-code login, admin panel, hospital ya blood bank features, aur server-side logout endpoint (JWT stateless hai, logout me client token delete karta hai).

SAB

QUICK AUDIT - ek hi prompt me chhota audit

Kab chalana hai: Jab time kam ho. Poora audit chahiye to A0 se A10 use karo, ye nahi.

- Ye sirf ek jaldi wali screening hai; deep checks (concurrency, e2e) isme nahi hain.
- Result me NOT-BUILT-YET wali cheezein bug nahi hain, unhe ignore karo.

```
## ROLE
You are a senior software auditor (backend, mobile, security, QA). Audit the LifeDrop project in this
workspace. This is an AUDIT, not a build. I have finished build prompts up to: <N>.

## PROJECT
LifeDrop connects ONLY DONOR and RECIPIENT users (no hospitals, no blood banks, no OTP login, no admin).
Flow: recipient creates a blood request or SOS -> compatible nearby donors are matched (blood group +
Haversine distance + 90-day gap + available + active) -> donors are notified -> first donor to accept
wins -> phone numbers revealed only after accept -> recipient marks completed -> donor
lastDonatedDate = today.
Stack: Spring Boot 3 / Java 17 / PostgreSQL / JWT (lifedrop/backend), React + Vite (lifedrop/web),
React Native Expo SDK 57 (lifedrop/mobile). Missing folders must be reported.

## AUDIT RULES (apply to every audit step)
1. READ-ONLY on source code. You may only (a) create or update AUDIT_REPORT.md at the repo root,
   (b) add NEW test files under lifedrop/backend/src/test or audit-tests/, (c) create normal build
   output (target/, node_modules/, .expo/). If package-lock.json changes after npm install, tell me.
2. Work on a git branch named audit/<today's date>. Never commit to main.
3. Any test that writes data (create users, requests, parallel accepts, e2e script) must run against
   a SEPARATE database: lifedrop_audit (PostgreSQL) or the H2 test profile. NEVER touch my real
   lifedrop database.
4. Evidence over opinion. Every finding needs a file path with line numbers, or the exact command you
   ran with its shortened output.
5. Use exactly these statuses:
   VERIFIED       = you ran it (or a test proved it) and it passed
   FAILED         = you ran it and it failed
   CODE-REVIEW-ONLY = you read the code but could not run it
   NOT-VERIFIED   = you could not check it (say why)
   MISSING        = it should exist at my current build stage but does not
   NOT-BUILT-YET  = it belongs to a build prompt I have not run yet (this is NOT a bug)
6. Never print secrets (passwords, JWT secret, API keys, tokens, push tokens). Mask them as ****.
7. If you cannot run something (database down, no device), say so and continue with code review.
   Never guess.
8. Severity: P0 = security hole, data loss, or core flow broken. P1 = important bug or missing
   feature in a main flow. P2 = minor bug, quality, polish. P3 = suggestion.
9. Do NOT fix anything. Fixes happen only after I approve the report (Prompt F1).

## WHAT TO AUDIT
A. Feature coverage vs build prompts 1 to 17 (Done / Partial / Missing / NOT-BUILT-YET).
B. Build and config: .\mvnw.cmd clean test, app start with the dev profile, hard-coded secrets and
   default JWT secret, ddl-auto, CORS, Swagger exposure, leftover test endpoints.
C. Security: call the running API with no token / wrong role / another user's resource (IDOR);
   phone, email and coordinates must never leak before accept; mass assignment on PUT /api/users/me.
D. Business logic with tests: 64 blood-compatibility pairs, Haversine, 90-day boundary (89/90/91),
   20 parallel accepts (exactly one must win; if no lock or @Version, that is P0), forbidden status
   transitions, rate limit (4th request returns 429), expiry job, report auto-cancel after 3 reports.
E. Mobile and web: token storage, 401 handling, route guards, hard-coded URLs, loading/empty/error
   states, content check (no fake claims, no certification words).
F. Production readiness: secrets in git history (public repo, never print them), prod profile.

## OUTPUT (AUDIT_REPORT.md)
1. Summary: counts of P0/P1/P2/P3 and a 5-line verdict: ready for real users? yes / no / closed pilot.
2. "What works (VERIFIED)" list.
3. Findings table: ID | Area | Finding | Severity | Status | Evidence | Recommended fix.
4. Prioritized fix plan (P0 first) with effort S/M/L.
Finish by telling me the 3 most dangerous problems in plain Hinglish. Do not start fixing.
```

SETUP

PROMPT A0 - Master Audit Context (sabse pehle ye bhejo)

Kab chalana hai: Naye audit chat me sabse pehla message. Isme audit nahi hota, sirf rules set hote hain.

- <N> ki jagah apna last finished build prompt number likho (jaise 10).
- Pehle git branch banao: git checkout -b audit/2026-10-21 (apni date daalo).
- PostgreSQL me alag database banao: CREATE DATABASE lifedrop_audit;
- Agar Antigravity me Rules / Workspace instructions hai to ye A0 wahan bhi save kar do.

```
## ROLE
You are a senior software auditor (backend, mobile, security, QA). You will audit the LifeDrop project
in this workspace in several steps. This is an AUDIT, not a build.

## PROJECT SUMMARY
LifeDrop connects ONLY two user types: DONOR and RECIPIENT. There are no hospitals, no blood banks, no
trauma centers and no admin-managed inventory. Flow: a recipient creates a blood request (or SOS) ->
compatible nearby donors are matched (blood-group compatibility + Haversine distance + 90-day gap +
available + active) -> donors are notified (in-app + Expo push) -> first donor to accept wins ->
phone numbers are revealed only after accept -> recipient marks the request completed -> the donor's
lastDonatedDate becomes today.
Stack: Spring Boot 3.x (Java 17, Maven), PostgreSQL, JWT + BCrypt in lifedrop/backend; React 18 + Vite
+ Tailwind in lifedrop/web; React Native Expo SDK 57 in lifedrop/mobile.

## BUILD STAGE
The project was built with build prompts 0 to 17. I have finished build prompts up to: <N>.
Anything that belongs to a later prompt is NOT-BUILT-YET, not a bug.

## BUILD PROMPT MAP (what each prompt was supposed to deliver)
1  backend setup (health endpoint, CORS, env-based config)
2  entities: User, BloodRequest, RequestResponse, Notification (+ indexes, unique request+donor)
3  auth: register / login / me, BCrypt, JWT, roles, clean JSON 401/403, GlobalExceptionHandler
4  profile, location, push-token, donor availability, 90-day eligibility endpoint
5  BloodCompatibility, DistanceCalculator, nearby donor native SQL, /api/recipient/donors/nearby
6  request flow: create / list / details / cancel / complete, donor accept / reject / history,
   @Scheduled expiry job (24h), first accept wins (409)
7  notifications: in-app rows + Expo push, list / unread-count / read / read-all
8  safety: report endpoint (3 reports cancel the request), 3 requests per 24h limit, dev seed data,
   Swagger with Authorize, tests
9  web: theme, navbar, landing page        10 web: auth, dashboards, map, other pages
11 mobile: setup, theme, navigation, auth  12 mobile donor screens   13 mobile recipient + map
14 mobile: location sync, push, notification screen    15 SOS (backend + web + mobile)
16 QA + polish                             17 Dockerfile, prod profile, eas.json, privacy, delete
   account

## NOT PART OF THIS PROJECT (never report these as missing)
OTP / email-code login, admin panel, hospital or blood-bank features, a server-side logout endpoint
(JWT is stateless; logout means the client deletes the token).

## AUDIT RULES (apply to every audit step)
1. READ-ONLY on source code. You may only (a) create or update AUDIT_REPORT.md at the repo root,
   (b) add NEW test files under lifedrop/backend/src/test or audit-tests/, (c) create normal build
   output (target/, node_modules/, .expo/). If package-lock.json changes after npm install, tell me.
2. Work on a git branch named audit/<today's date>. Never commit to main.
3. Any test that writes data (create users, requests, parallel accepts, e2e script) must run against
   a SEPARATE database: lifedrop_audit (PostgreSQL) or the H2 test profile. NEVER touch my real
   lifedrop database.
4. Evidence over opinion. Every finding needs a file path with line numbers, or the exact command you
   ran with its shortened output.
5. Use exactly these statuses:
   VERIFIED       = you ran it (or a test proved it) and it passed
   FAILED         = you ran it and it failed
   CODE-REVIEW-ONLY = you read the code but could not run it
   NOT-VERIFIED   = you could not check it (say why)
   MISSING        = it should exist at my current build stage but does not
   NOT-BUILT-YET  = it belongs to a build prompt I have not run yet (this is NOT a bug)
6. Never print secrets (passwords, JWT secret, API keys, tokens, push tokens). Mask them as ****.
7. If you cannot run something (database down, no device), say so and continue with code review.
   Never guess.
8. Severity: P0 = security hole, data loss, or core flow broken. P1 = important bug or missing
   feature in a main flow. P2 = minor bug, quality, polish. P3 = suggestion.
9. Do NOT fix anything. Fixes happen only after I approve the report (Prompt F1).

## REPORT FORMAT (AUDIT_REPORT.md)
```

```
Keep ONE report file. Every finding is a table row:
ID | Area | Finding | Severity | Status | Evidence (file:line or command) | Recommended fix
Keep two summary sections at the top: "What works (VERIFIED)" and "What is missing or broken".

## ENVIRONMENT
I use Windows PowerShell. Use .\mvnw.cmd, npm.cmd, npx.cmd and curl.exe.

## NOW
Reply only with: (a) the folders you found, (b) the exact commands you will use to build, run and
test each part, (c) anything you need from me (for example: PostgreSQL running, DB_PASS set, the
lifedrop_audit database created, which build prompts are finished). Do not start the audit yet.
```

SAB

PROMPT A1 - Feature Coverage (kya hai, kya nahi hai)

Kab chalana hai: A0 ke baad. 'Kya kami hai' ka seedha jawab. Kabhi bhi chala sakte ho, jitna build hua hai utna check hoga.

- NOT-BUILT-YET wali cheezein Top 10 missing list me mat aani chahiye; wo tumhare agle build prompts hain.

```
## TASK
Audit 1: Feature coverage. Compare the code with the feature list below and tell me exactly what
exists, what is partial, what is missing and what is NOT-BUILT-YET. Read-only.

## FEATURE LIST (check backend, web and mobile separately)
AUTH (build prompt 3): POST /api/auth/register (DONOR or RECIPIENT only; donor needs bloodGroup and age
  18 to 65; phone 10 digits; password min 8), POST /api/auth/login, GET /api/auth/me, BCrypt, JWT,
  role-based routes (/api/donor/** DONOR only, /api/recipient/** RECIPIENT only), JSON 401/403.
PROFILE (4): GET/PUT /api/users/me (never email or role), PUT /api/users/location (range checks),
  PUT /api/users/push-token, PUT /api/donor/availability (ON refused while ineligible, with the next
  eligible date), GET /api/donor/eligibility.
MATCHING (5): BloodCompatibility, DistanceCalculator, native nearby SQL, GET
   /api/recipient/donors/nearby
  (first name only, no phone / email / exact coordinates).
REQUEST FLOW (6): recipient create / list / details / cancel / complete; donor incoming list /
  details / accept / reject / history; 10 km NORMAL and 25 km URGENT radius; one RequestResponse per
  matched donor; @Scheduled expiry of PENDING requests older than 24h.
NOTIFICATIONS (7): in-app row always, Expo push when pushToken exists, push failure never breaks the
  main flow; GET /api/notifications (paged), unread-count, read, read-all.
SAFETY (8): POST /api/requests/{id}/report (no double report, 3 reports cancel), 429 after 3 requests
  per 24h, dev-profile seed (12 Bhopal donors + 2 recipients), Swagger with Authorize.
WEB (9, 10): landing page, navbar, footer, login / register, donor dashboard, recipient dashboard,
  find donors map (Leaflet), blood requests page, profile, notifications dropdown, privacy, terms, 404.
MOBILE (11 to 14): splash, login, register, role tabs, donor home / requests / request detail /
  history, recipient home / create request / nearby map / my requests / request status, profile,
  location sync (donor, max once per 30 min), push token + channels, notification screen.
SOS (15): POST /api/recipient/sos (25 km, fallback 50 km, one active SOS, requiredBy now + 6h), mobile
  SOS sheet + status screen with 112 / 108 buttons, web SOS modal.
RELEASE (17): Dockerfile, application-prod.properties, restricted CORS, eas.json (development, preview,
  production), app.json fields, privacy page, DELETE /api/users/me (anonymizes data).

## HOW TO JUDGE
Done = complete AND wired end to end: UI calls API, API calls service, service saves to the database,
the result comes back to the UI. Partial = one link of that chain is missing (say which). Missing =
should exist at my build stage but there is no code. NOT-BUILT-YET = belongs to a prompt after <N>.

## OUTPUT (AUDIT_REPORT.md, section "1. Feature coverage")
1. Table: Feature | Build prompt | Backend | Web | Mobile | Evidence (file:line) | Notes.
2. Extra things in the code that are NOT in my list (extra endpoints, screens, test endpoints such as
   /api/donor/test or /api/recipient/test).
3. Dead code: endpoints no UI calls, screens not reachable from navigation, unused components.
4. Hard-coded demo or fake data in web or mobile screens (lists, counts, sample requests).
5. A short list "Top 10 missing things, most important first" (exclude NOT-BUILT-YET).
```

BACKEND

PROMPT A2 - Backend Build, Startup and Config

Kab chalana hai: Build prompt 8 ke baad (backend complete). PostgreSQL chalu aur lifedrop_audit database bana hua ho.

- Tests me H2 aur startup me dev profile use hoga; real lifedrop database ko haath nahi lagana.

```
## TASK
Audit 2: Backend build, startup and configuration. Source files are read-only.

## DO THESE
1. In lifedrop/backend run .\mvnw.cmd clean test. Report compile errors, failing tests, skipped tests,
   total test count, and which areas have NO tests at all (auth, profile, notifications, scheduler,
   report, rate limit, expiry job).
2. Start the app against the lifedrop_audit database with the dev profile (DB_PASS is in my
   environment). Report startup time, warnings, ERROR logs, deprecation warnings.
3. List every property in application*.properties and say whether it reads from an environment
   variable. Report any secret or default password that is hard-coded or has a default value in code,
   properties, .vscode or Docker files. In particular check whether JWT_SECRET has a hard-coded
   fallback: a fallback is acceptable for dev only and must NOT exist in application-prod. Check the
   seed password (Password@123) is created only under the dev profile.
4. Search git history for secrets: git log --oneline -S"password", -S"secret", -S"apiKey" and look at
   .env files that were ever committed. The repository is PUBLIC. Never print the value; only say
   which commit and file, and that the secret must be rotated.
5. Compare JPA entities with the real PostgreSQL schema (psql or SQL queries): indexes (users role /
   blood_group / available, blood_requests status), NOT NULL, unique users.email, unique
   request_responses(request_id, donor_id), foreign keys, column types for dates / timestamps,
   latitude / longitude precision.
6. Check: spring.jpa.hibernate.ddl-auto (update is a P1 for production), open-in-view, show-sql,
   Swagger exposure, actuator exposure, CORS config (allow-all is only OK for dev), server.address,
   default error page, logging level, leftover test endpoints (/api/donor/test, /api/recipient/test).
7. pom.xml: outdated or vulnerable dependencies; is Spring Boot 3.2.x still supported? (verify the
   current support status, do not guess). Check jjwt version and Java 17 usage. Report only.

## OUTPUT
AUDIT_REPORT.md section "2. Backend build and config" with the findings table, plus the test result
numbers (run / failed / skipped).
```

BACKEND

PROMPT A3 - Security and Access Control

Kab chalana hai: A2 ke baad. Backend (lifedrop_audit DB par) chal raha hona chahiye; ye asli API call karke test karta hai.

- Yahan OTP wala section nahi hai kyunki project me OTP login hai hi nahi.
- Ye step production data par kabhi mat chalana.

```
## TASK
Audit 3: Security and access control. TEST by calling the running API (curl.exe or a script), not only
by reading code. Source files are read-only. Use the lifedrop_audit database only.

## BUILD A TEST MATRIX
Create donorA, donorB, recipientA, recipientB. For EVERY endpoint call it with: (1) no token,
(2) invalid token, (3) tampered token (change one character), (4) donor token, (5) recipient token,
(6) another user's resource id. Record the HTTP status and compare with what it SHOULD be.
Endpoints: /api/auth/*, /api/users/*, /api/donor/*, /api/recipient/*, /api/notifications/*,
/api/requests/{id}/report, /api/health, Swagger URLs, and the SOS and delete-account endpoints if built.

## CHECK SPECIFICALLY
AUTH
- Password is hashed with BCrypt, never returned in any response, never logged.
- Wrong email and wrong password give the same message (no user enumeration). Register with an
  existing email gives a clear 409 or 400, not a 500.
- JWT: algorithm, secret length (at least 32 bytes) and source, expiry, tampered or expired token is
  rejected, role is read from the server-side user (not trusted blindly), token never logged.
- Brute force: is there any lockout or rate limit on login? The build prompts did not ask for one, so
  report it as a finding (P1 or P2) with a recommendation, not as a defect of the prompts.
- Register: only DONOR or RECIPIENT is accepted (no other role); donor age outside 18 to 65 is
  rejected; donor without bloodGroup is rejected.
AUTHORIZATION (IDOR)
- Recipient A cannot read, cancel, complete or report-manage requests of recipient B.
- A donor cannot read or accept a request that was not matched to him, cannot accept twice, cannot
  act after another donor accepted.
- A user cannot read or mark-as-read another user's notification (PUT /api/notifications/{id}/read).
- Who may call POST /api/requests/{id}/report? Can the owner report their own request? Can anyone
```

```
  report any request id? Can a report flip a COMPLETED request to CANCELLED? Test and report.
PRIVACY
- The other party's phone and email appear ONLY after accept and ONLY to the two parties involved.
  Check every endpoint and DTO that could leak phone, email, exact coordinates, pushToken or
  lastDonatedDate: nearby donors, incoming requests, history, notifications, error messages, Swagger
  examples.
MASS ASSIGNMENT
- PUT /api/users/me must not change role, email, password, active, lastDonatedDate, id, pushToken.
- PUT /api/users/location must reject out-of-range values. PUT /api/donor/availability must refuse ON
  while the donor is ineligible.
INPUT
- SQL injection in the native Haversine query and every @Query; validation on every DTO; oversized
  payloads; stored XSS in message, patientName and report reason (check how the web app renders them);
  invalid enum values must return 400, not 500; negative or huge unitsNeeded; requiredBy in the past.
TRANSPORT AND CONFIG
- CORS is not "*" for production; security headers; Swagger not public in prod; stack traces never
  returned to the client; the GlobalExceptionHandler fallback hides internal messages.
DATA
- Personal data in logs (phone, tokens, request bodies); push token exposure; whether the delete-account
  endpoint (build prompt 17, if built) really anonymizes name, phone, email, location and pushToken.

## OUTPUT
Section "3. Security": the endpoint matrix with columns Endpoint | Method | Expected | No token |
Bad token | Donor | Recipient | Other user, and the findings table.
Put every P0 at the very top of the section.
```

BACKEND

PROMPT A4 - Business Logic Correctness (sabse zaroori)

Kab chalana hai: Build prompt 8 ke baad. Matching, accept, status, 90-din rule sab yahin prove hota hai. Isme tests likhe jaate hain.

- 20 parallel accept wala test PostgreSQL (lifedrop_audit) par chalana chahiye; H2 ka locking behaviour alag ho sakta hai.
- Agar accept me koi lock ya @Version nahi mila to ye seedha P0 hai.

```
## TASK
Audit 4: Business logic. Prove EVERY rule below with a real test (JUnit / integration tests or API
calls). You may add NEW test files only under lifedrop/backend/src/test. Do not change production
code. Run concurrency tests against PostgreSQL (lifedrop_audit), not only H2.

## RULES TO PROVE
1. BLOOD COMPATIBILITY: test all 8 recipient groups against all 8 donor groups (64 pairs):
   A+ <- A+, A-, O+, O- | A- <- A-, O- | B+ <- B+, B-, O+, O- | B- <- B-, O-
   AB+ <- all 8 | AB- <- AB-, A-, B-, O- | O+ <- O+, O- | O- <- O- only.
2. DISTANCE: Haversine vs known pairs (Indore 22.7196,75.8577 to Bhopal 23.2599,77.4126 is about
   170 km straight line, tolerance 5 km; same point = 0). Inspect the native SQL for the
   acos() domain problem (a value like 1.0000000002 gives NaN): it must be clamped with LEAST/GREATEST.
   Also test: null latitude/longitude, donor exactly at the radius, radius 0 / negative / huge, and
   performance with 10,000 donors (EXPLAIN ANALYZE; is there a bounding-box prefilter or index?).
3. ELIGIBILITY: lastDonatedDate = null, 89 days, exactly 90 days, 91 days, a future date. Check which
   clock and time zone LocalDate.now() uses. The production host (Render) runs in UTC while users are
   in India: show whether a donor can be off by one day near midnight IST.
4. AVAILABILITY: unavailable or inactive donors are never matched. A donor cannot switch ON while
   ineligible (400 with nextEligibleDate).
5. REQUEST CREATION: one RequestResponse per matched donor; the requester is never matched with himself
   (check that a recipient's own account can never appear in the donor list); zero matches gives
   a clear message; radius 10 km for
   NORMAL and 25 km for URGENT unless radiusKm is given; max 3 requests per 24 hours (4th returns 429);
   invalid blood group, unitsNeeded, requiredBy in the past are rejected.
6. ACCEPT (CONCURRENCY): fire 20 parallel accept calls from 20 different matched donors for one request.
   Exactly ONE must succeed, the others must get 409, and the database must show exactly one
   acceptedDonor and exactly one ACCEPTED RequestResponse. State which locking is used (pessimistic
   lock, @Version, or a conditional UPDATE). If none is used, this is a P0.
7. STATUS MACHINE: write down all allowed and forbidden transitions as the code implements them, then
   test the forbidden ones: accept a CANCELLED request, accept an EXPIRED request, complete a PENDING
   request, complete twice, cancel after COMPLETED, cancel an EXPIRED request, reject after accept,
   accept twice by the same donor. Final states: COMPLETED, CANCELLED, EXPIRED.
8. CANCEL AFTER ACCEPT: when a recipient cancels an ACCEPTED request, is the accepted donor told? Does
   the donor's pending-requests list and history stay correct? Report what happens.
9. COMPLETE: only the owner recipient can complete; lastDonatedDate = today is set for the ACCEPTED
   donor only; the donor becomes ineligible for 90 days; history shows the donation once (double tap
   must not duplicate).
10. EXPIRY JOB: the @Scheduled job (every 15 min) marks PENDING requests older than 24h EXPIRED and
    their PENDING responses too; running it twice is safe; ACCEPTED requests are never expired. Also:
```

```
    is expiry enforced when a request is READ or ACCEPTED (so a sleeping or restarted server does not
    leave an old request acceptable)? Test accept on a 25-hour-old PENDING request.
11. REPORTS: a user cannot report the same request twice; the 3rd distinct report cancels the request
    and notifies pending donors; check who can report and which statuses can be flipped.
12. PRIVACY: before accept no phone number anywhere; after accept visible only to the two parties.
13. NOTIFICATIONS: one in-app row and one push attempt per event (new request, accepted, cancelled,
    completed); a push failure never breaks the main transaction; no duplicates; notifications are
    created only if the transaction commits (not before a rollback).
14. TRANSACTIONS: multi-step operations are @Transactional and roll back fully on error (simulate a
    failure in the middle of create-request and in accept).
15. SOS (only if build prompt 15 is done): 25 km then 50 km fallback; one active SOS per recipient;
    requiredBy = now + 6h; response contains requestId and donorsNotified.

## OUTPUT
Section "4. Business logic": table Rule | Test written | Result (PASS / FAIL) | Evidence, then the
findings table. List every test file you added and the command to run them again.
```

BACKEND

PROMPT A5 - API Contract and Database

Kab chalana hai: A4 ke baad.

- Flyway ka sirf recommendation aur V1 ka content dikhana hai; file abhi banani nahi hai.

```
## TASK
Audit 5: API contract and data layer. Read-only.

## API
- Response format and HTTP status codes are consistent (ApiResponse wrapper everywhere or not);
  one error JSON shape for 400 / 401 / 403 / 404 / 409 / 429 / 500; readable field-wise validation
  messages.
- Pagination: build prompt 7 asked for paged notifications only. Check which other list endpoints
  (recipient requests, donor incoming requests, history, nearby donors) can grow without a limit and
  report them with a recommended maximum page size.
- Entities returned directly instead of DTOs; LazyInitializationException risks (open-in-view is
  false); N+1 queries: turn SQL logging on for one test run and count the queries for
  GET /api/recipient/requests, GET /api/donor/requests and GET /api/notifications. (Example: the
  recipient request list counts responses once per request.)
- JSON naming is consistent (camelCase); dates are ISO-8601; enum values are documented in Swagger.
- Swagger matches reality: run each documented example. List duplicate, unused or test endpoints.

## DATABASE
- Missing indexes for the real queries: nearby search, request_responses by donor and status,
  request_responses by request, notifications by user and read flag, blood_requests by recipient and
  createdAt.
- Cascade and orphan rules: what happens to requests, responses and notifications when a user is
  deleted or anonymized (build prompt 17 anonymizes instead of deleting); unique constraints survive?
- Migration strategy: ddl-auto=update is NOT safe for production. Recommend Flyway and show, as text
  only, what migration V1 would contain. Do not create any file.
- Column types for money-free data: timestamps (with or without time zone), latitude and longitude
  (double vs numeric), and text length limits for message and report reason.

## OUTPUT
Section "5. API and data" with the findings table and the "V1 migration outline".
```

MOBILE

PROMPT A6 - Mobile App (React Native, Expo)

Kab chalana hai: Build prompt 14 ke baad (mobile screens, push, notification screen). Pehle tak jo bana hai utna check hoga.

- Push notification real phone par development build se test hota hai; Expo Go me remote push ka support alag hota hai (SDK 57 ke liye apni docs se verify karo).
- npm audit sirf report ke liye; npm audit fix --force kabhi nahi.

```
## TASK
Audit 6: React Native (Expo SDK 57) app in lifedrop/mobile. Source files are read-only.

## RUN
npm.cmd install, npx.cmd expo-doctor, a bundle / export check to confirm it compiles, lint if
configured. Report warnings, SDK or version mismatches (everything must match Expo SDK 57), unused
dependencies, vulnerable packages. Report only. NEVER run npm audit fix --force.

## CODE REVIEW CHECKLIST (build prompts 11 to 14, and 15 / 17 items only if built)
CONFIG: EXPO_PUBLIC_API_URL is the only API base (the 10.0.2.2 and localhost values are acceptable only
as dev defaults; a release build must use https); no secret in EXPO_PUBLIC_ variables; app.json is
valid; Google Maps key is a placeholder, not a real key in git; permissions; android package; icon
and splash.
AUTH: token lives only in expo-secure-store (never AsyncStorage, never logged); app restart restores
the session; a 401 or expired token logs out and goes to Login; logout clears token, user, push token
and cached data; DONOR never sees recipient tabs and the reverse; Register shows donor-only fields for
DONOR; client-side validation matches backend rules (age 18 to 65, phone 10 digits, password min 8).
NETWORK: axios timeout; friendly messages for no internet, server down, 4xx and 5xx; no unhandled
promise rejections; loading flags reset in finally blocks; double-tap protection on Submit, Accept,
Reject, Complete and SOS buttons; the 409 "another donor already accepted" message.
SCREENS: every list has loading, empty and error states and pull-to-refresh; FlatList with
keyExtractor; no state update after unmount; polling (SOS status every 10s, notifications) is cleaned
up on unmount and paused in background; keyboard avoiding, safe areas, Android back button; layouts at
360 px width.
LOCATION: permission reason text shown before the system dialog; denied and "never ask again" paths;
GPS off; timeouts; donor location update at most once per 30 minutes (useLocation); map with 100+
markers; city fallback when permission is denied.
PUSH: Android 13+ POST_NOTIFICATIONS request; token sent to PUT /api/users/push-token after login and
cleared on logout; channels "default" and "urgent"; notification tap opens the right screen even when
the app was closed; foreground banner.
PRIVACY AND SAFETY: phone number appears only after accept; Call and WhatsApp use Linking safely
(encode the number); medical disclaimer visible; no personal data in console logs.
CONTENT: no wording about hospitals, blood banks, certifications or fake statistics (the disclaimer
"contact a hospital or call 112" is allowed).
DESIGN SYSTEM: colors, Plus Jakarta Sans weights, radius (buttons 12, cards 20, pill badges), red glow
shadow, StatusBadge colors, contrast of muted text on #FFF5F5.
QUALITY: files over 300 lines, duplicated code, TODO / FIXME list, accessibility labels on buttons.

## OUTPUT
Section "6. Mobile app": findings table, list "Screens reachable from navigation", list of missing
loading / empty / error states, and a list of screens named in build prompts 11 to 14 that do not
exist yet (mark them NOT-BUILT-YET or MISSING according to <N>).
```

WEB + UI

PROMPT A7 - Web App and UI / Theme

Kab chalana hai: Build prompt 10 ke baad (ya 15 ke baad SOS modal ke saath).

- Content check me required disclaimer ('contact a hospital or call 112') violation nahi maana jaata.

```
## TASK
Audit 7: Web app (lifedrop/web) and the design system. Source files are read-only. If a folder is
missing, say so.

## RUN
npm.cmd install, npm.cmd run build, npm.cmd run lint (if present), npm.cmd audit (report only, never
--force). Report bundle size and warnings.

## WEB CHECKS
- Route guards for guest / donor / recipient (ProtectedRoute): try each protected URL as guest and as
  the wrong role.
- Token storage: localStorage is XSS-exposed; list where user-supplied text (message, patientName,
  report reason, notification body) is rendered and confirm it is not injected as HTML.
- axios interceptor adds the JWT; a 401 logs out and redirects to /login; VITE_API_URL is the only API
  base (no hard-coded localhost).
- Forms validate before submit and show backend field errors; skeleton loaders, empty states and toasts
  exist.
- Open every page and read the browser console for errors and warnings.
- Responsive at 360, 768 and 1280 px; hamburger menu on mobile; keyboard access; alt text.
- Leaflet map loads (its CSS is imported) and donor pins never show phone numbers.
- Deep-link refresh (needs a rewrite to index.html on the host); /privacy, /terms and 404 pages;
  page title and meta description.
- Notifications dropdown polls every 30s and cleans up its interval.
- SOS modal (build prompt 15, only if built): logged-in recipient opens the flow, guest is sent to
  Register.
- Pages that show hard-coded demo data, mock requests or fake counts (stats strip must show 0 until API
  data exists).

## DESIGN SYSTEM CHECK
Tokens: primary-dark #991B1B, primary #B91C1C, accent/SOS #DC2626, SOS gradient end #BE123C, page
background #FFF5F5, soft pink #FEE2E2, border #FECACA, ink #0F172A, muted #475569, light-muted #94A3B8,
success #16A34A, warning #F59E0B. Font Plus Jakarta Sans 400 to 800; headings weight 800 with tight
tracking. Radius: buttons 12 px, cards 20 px, pill badges. Glow shadow 0 10px 30px
rgba(185,28,28,0.30) on primary and SOS buttons. Find hard-coded colors outside the theme, wrong
fonts, inconsistent buttons, missing status colors, low-contrast text.

## CONTENT CHECK (important)
Search the whole web and mobile UI for: hospital, blood bank, trauma, certified, NABH, CDSCO, "free",
"guaranteed", "100%", and numbers that look like statistics ("10,000+ donors"). Rules: the medical
disclaimer "LifeDrop connects donors and recipients. In a medical emergency, contact a hospital or call
112." must be visible and is allowed. Marketing lines such as "No hospitals, no blood banks", "Zero
hospital fees" or "bypassing hospital queues", and form labels like "Hospital Location", are listed as
P2 content findings (Master Context says never add hospital or blood-bank wording). Also check the
Privacy Policy text makes no false legal claims.

## OUTPUT
Section "7. Web and UI": findings table and a list of every screen that violates the design system
or the content rules.
```

FULL-STACK

PROMPT A8 - End-to-End Scenario Test (asli flow chalake)

Kab chalana hai: Build prompt 15 ke baad (SOS ke saath). Backend lifedrop_audit DB par chalu rakho.

- Script apne test users banata hai aur end me clean karta hai; real DB par kabhi nahi.

```
## TASK
Audit 8: End-to-end scenarios. Write ONE script (audit-tests/e2e.ps1 or e2e.js) that runs the scenarios
below against the running API (lifedrop_audit database) and prints PASS / FAIL for each. It creates its
own users and cleans them up at the end. Do not change source files.

## SCENARIOS
S1  Register donor age 17 -> 400.     S2  Register donor age 66 -> 400.
S3  Register donor without bloodGroup -> 400.
S4  Register the same email twice -> clear 409 or 400, not 500.
S5  Login with a wrong password -> 401 and the same message as an unknown email.
S6  Recipient (A+) creates a NORMAL request: donors A+, A-, O+, O- within 10 km are matched; a B+
    donor is NOT.
S7  A donor 30 km away is NOT matched for NORMAL (10 km) but IS matched for URGENT (25 km).
S8  An unavailable donor and an inactive donor are NOT matched.
S9  A donor who donated 30 days ago is NOT matched; one who donated 91 days ago IS matched.
S10 Before accept: no phone number in any response for either user (check JSON of every endpoint).
S11 Two donors accept at the same time: exactly one succeeds, the other gets 409.
S12 After accept: recipient sees the donor phone; donor sees the recipient phone; a third matched
    donor sees neither.
S13 Recipient cancels a PENDING request: donors can no longer accept; pending donors got a notification.
S14 Recipient cancels an ACCEPTED request: report what happens to the accepted donor (is he notified?).
S15 Recipient completes an ACCEPTED request: donor history updated, donor ineligible for 90 days,
    availability ON is refused with the next eligible date.
S16 Complete twice -> second call rejected, no duplicate history.
S17 Recipient B opens recipient A's request id -> 403 or 404. A donor token on a recipient endpoint ->
   403.
S18 The 4th request within 24 hours -> 429.
S19 Report: 3 different users report one request -> it becomes CANCELLED; the same user reporting twice
    is rejected.
S20 A 25-hour-old PENDING request cannot be accepted (expired).
S21 Notifications: each event creates exactly one in-app notification per target user; unread-count and
    mark-read / read-all work; user A cannot mark user B's notification.
S22 Expired or tampered token -> 401 and a fresh login works.
S23 SOS (only if built): urgent request created, 25 km then 50 km fallback, a second active SOS is
    rejected, response has requestId and donorsNotified.
S24 Delete account (only if built): personal data anonymized, old token no longer works, the email can
    be reused or is blocked consistently.

## OUTPUT
Run the script, paste the result table (Scenario | Expected | Actual | PASS / FAIL) into AUDIT_REPORT.md
section "8. End-to-end", and tell me the exact command to run it again.
```

RELEASE

PROMPT A9 - Production Readiness

Kab chalana hai: Build prompt 17 ke baad, live karne se pehle. Isse pehle chalaoge to bahut saare NOT-BUILT-YET aayenge.

- Repo public hai: agar kabhi koi secret commit hua tha to usse rotate karna hi padega (git history badalna kaafi nahi).

```
## TASK
Audit 9: Production readiness for build prompt 17 (Render or Railway backend, Neon or Supabase
PostgreSQL, EAS Android build). Read-only.

## CHECK
SECRETS: any secret in code, properties, .vscode, Docker files or git history (git log -S; never print
the value). The repository is public: if something was ever committed, say so and say it must be
rotated. JWT_SECRET must come only from the environment in the prod profile (no default value).
CONFIG: application-prod.properties exists and reads everything from environment variables; Swagger
and the dev seed data are OFF in prod; ddl-auto is not "update" (validate or none, with Flyway);
CORS lists only the real website; forwarded headers (X-Forwarded-*) handled behind the proxy; show-sql
false; error responses hide stack traces; SPRING_PROFILES_ACTIVE=prod works.
DOCKER: Dockerfile is multi-stage on Java 17, builds, runs as a non-root user, and the container
starts with only the environment variables listed in the build prompt (DB_URL, DB_USER, DB_PASS,
JWT_SECRET, JWT_EXPIRATION, SPRING_PROFILES_ACTIVE).
RELIABILITY: health endpoint; timeouts for outgoing Expo push calls; connection pool size for the free
database tier; thread-safe @Scheduled job; free-tier hosts sleep: the 15-minute expiry job will not run
while asleep, so check that expiry is also enforced at read and accept time; a time-zone plan (server is
UTC, users are in India, 90-day rule and "today" must be consistent); backup and restore plan for
PostgreSQL (propose commands).
LOGGING AND PRIVACY: no phone, email, token or push token in logs; log levels; what personal data is
stored (name, phone, blood group, location); DELETE /api/users/me really anonymizes (name, phone,
email, location, pushToken, password) and keeps requests consistent; data retention for old requests.
LEGAL AND STORE: privacy policy at /privacy (data collected, why, who sees it, deletion, contact email
placeholder replaced by a real address); medical disclaimer; Play Console data-safety answers match
the real data; the store description has no false claims (no certification, no hospital or blood-bank
claims, no fake numbers).
MOBILE BUILD: eas.json has development, preview (APK) and production (AAB) profiles; app.json name
"LifeDrop", slug "lifedrop", version 1.0.0, android.package com.lifedrop.app, permissions
ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, POST_NOTIFICATIONS; EXPO_PUBLIC_API_URL is https for
preview and production; Google Maps key and push credentials (FCM) are documented but not committed.
DEPENDENCIES: vulnerable packages in backend, mobile, web (report only; never npm audit fix --force).

## OUTPUT
Section "9. Production readiness": findings table plus a Go / No-go checklist with Yes / No /
   NOT-BUILT-YET
for every item above.
```

SAB

PROMPT A10 - Final Report and Fix Plan

Kab chalana hai: Jitne audit chalane the wo ho jaane ke baad. Ye saari findings ko ek plan me jodta hai.

- Output ke baad ruk jao aur report khud padho; F1 me sirf approve ki hui IDs hi fix hongi.

```
## TASK
Audit 10: Consolidate everything. Do NOT fix anything.

## DO
1. Re-read AUDIT_REPORT.md. Remove duplicates, merge related findings, give every finding a unique ID
   (for example SEC-01, LOG-02, API-03, MOB-04, WEB-05, REL-06), a severity, a status and evidence.
2. Re-check each P0 and P1 yourself once more (run the command or test again) and mark it VERIFIED or
   FAILED. Downgrade anything you cannot prove.
3. Write a summary at the top:
   - counts of P0 / P1 / P2 / P3
   - "Ready for real users?" Yes / No / Only for a closed pilot, with 5 lines of reasoning
   - the 3 most dangerous problems in plain Hinglish
   - what is solid and well done
4. Fix plan in batches: Batch 1 = all P0, Batch 2 = P1 in main flows (auth, matching, accept,
   privacy), Batch 3 = other P1 and P2, Batch 4 = P3. For each item: ID, files to change, effort
   (S = under 1 hour, M = half a day, L = more), the test that will prove the fix, and the risk
   (data migration, breaking API change for mobile or web).
5. Mark items that need a decision from me (design choices, new dependencies, migrations, rotating
   secrets).
6. Run git status and git diff --stat. Confirm ONLY AUDIT_REPORT.md, audit-tests/ and new test files
   were added (plus build output). If any source file was modified, list it.

## OUTPUT
Update AUDIT_REPORT.md and give me a 15-line chat summary. Then wait for my approval.
```

FIX

PROMPT F1 - Fix approved findings

Kab chalana hai: Report padhne ke baad, sirf jo IDs tum approve karo unke liye.

- <IDS> me comma se IDs likho, jaise SEC-01, LOG-03. Ek baar me 3 se 5 hi rakho.
- Fixes naye branch fix/<batch> par karwao, main par seedhe nahi.

```
## TASK
I approved the audit report. Fix ONLY these findings, in this order: <IDS>.

## RULES
1. Work on ONE finding at a time. For each: (a) write or update a failing test that proves the problem,
   (b) make the smallest fix, (c) run the full test suite of that module, (d) show me a diff summary,
   (e) commit with the message "fix(<ID>): <short description>".
2. Do not refactor, rename or reformat code that is not part of the fix. Touch nothing outside the
   listed IDs.
3. If a fix needs a database migration, a new dependency, an API change that breaks the mobile or web
   app, or a design decision, STOP and ask me first.
4. After each batch (3 to 5 fixes) run: backend tests, npx.cmd expo-doctor, npm.cmd run build for web,
   and the e2e script in audit-tests/ against lifedrop_audit. Report PASS or FAIL.
5. Never print secrets. If a secret was found in git history, tell me the rotation steps but do NOT
   rewrite git history without my approval.
6. Update AUDIT_REPORT.md: set each fixed finding to FIXED with the commit id.

## OUTPUT
After each finding: ID, what was wrong, files changed, and the test that now passes. At the end: a
list of everything still open.
```

FIX

PROMPT R1 - Re-audit (fix ke baad dobara check)

Kab chalana hai: F1 ki batch khatam hone ke baad.

```
## TASK
Re-audit after fixes. Read-only again.

## DO
1. Re-run everything that was marked FAILED, plus the full backend test suite, the audit-tests/e2e
   script, npx.cmd expo-doctor and the web build.
2. For every finding marked FIXED in AUDIT_REPORT.md, re-test it independently. Mark it CONFIRMED-FIXED
   or REOPENED, with evidence.
3. Look for regressions: anything that passed before and fails now.
4. Look for new problems introduced by the fixes (new endpoints without security rules, new logs with
   personal data, new unvalidated inputs, new hard-coded values).
5. Update the summary: counts by severity before and after, and a new Go / No-go verdict.

## OUTPUT
A short table: ID | Before | After | Evidence. Then the remaining open items.
```